Pathology Voice Reporting — MVP

End-to-End Flow, State Management & Three-Milestone Implementation Plan

MVP scope: clinician voice → AssemblyAI streaming transcription → FastAPI/WebSocket → LangGraph incremental structured extraction and validation → cumulative report state → live UI updates → final clinician review → Supabase persistence. If the clinician does not dictate a comment, the AI generates one only during finalization.

CLINICIAN Speaks	→	**BROWSER** Microphone + WebSocket	→	**FASTAPI** Voice endpoint	→	**ASSEMBLYAI** Streaming STT
Continuous audio	→	Audio frames	→	Forwards audio	→	Meaningful/final transcript chunk
↓		↓		↓		
LANGGRAPH receive_chunk	→	**understand_chunk** LLM interprets chunk + context	→	**determine_updates** section + operation	→	**validate_update** Pydantic
New event enters graph state	→	Uses report state + active section + limited recent context + new text	→	Returns only changes; can target any section	→	Invalid → repair/retry route
↓		↓		↓		
merge_state Apply validated changes	→	**WebSocket event** Validated field/state update	→	**WAIT** Next transcript chunk	→	**FINISH** Clinician finalizes dictation
Cumulative report state	→	UI progressively fills report	Repeat chunk → graph → state → UI	Separate finalization path		
↓		↓		↓		
FINAL VALIDATION Validate complete report	→	**COMMENT DECISION** Clinician comment exists?	→	**YES** Keep clinician comment	/	**NO** Generate AI comment once
CLINICIAN REVIEW Edit / approve	→	**SUPABASE** Save finalized report	→	**COMPLETE** MVP report		

How the State Works

The report sections are not separate sequential LangGraph nodes. They live inside one cumulative state. Each new transcript chunk is processed against the current state and produces a validated delta. The delta is merged into the existing report rather than regenerating the entire report.

State	Purpose
transcript_chunk	Newest meaningful transcript from AssemblyAI.
report	Current structured report: patient, specimen, gross_description, microscopic_description, diagnosis, comments.
active_section	Routing/context hint; the LLM may override it when the new text clearly belongs elsewhere.
recent_context	Small local context window; do not send the entire transcript indefinitely.
updates	LLM-proposed structured changes for the current chunk.
validation_errors	Pydantic/business validation errors used for repair or rejection.

State evolution example

Chunk 1: “Gastric biopsy from the antrum.” → specimen populated.
Chunk 2: “Microscopically, chronic inactive gastritis.” → microscopic_description populated.
Chunk 3: “No Helicobacter organisms identified.” → appended to microscopic_description.
Chunk 4: “Final diagnosis is chronic inactive gastritis.” → diagnosis populated.
Finish: validate → check comment → preserve clinician comment OR generate one AI comment → review → save.

Core model: State(n+1) = Graph(State(n), TranscriptChunk(n))

Milestone 1 — Project Setup + AssemblyAI Integration

Objective: prove the complete real-time voice transport and transcription path before adding LLM/LangGraph logic.

Area	MVP work	Done when
Project setup	FastAPI app, config, .env, logging, Pydantic schemas, tests, clean folders.	Application starts cleanly and configuration is externalized.
WebSocket	Browser connects, sends audio, receives events; lifecycle/error handling.	Stable connect/send/receive/disconnect flow.
AssemblyAI	Streaming STT service; transcript event handling; cleanup/errors.	Speech reliably produces meaningful transcript chunks.
Supabase	Minimal cases/reports tables and repository layer.	Final-report persistence path exists; no need to save every live chunk.
Baseline UI	Mic control, connection status, live transcript display.	Doctor can speak and see transcription.

Milestone 1 output: Microphone → WebSocket → FastAPI → AssemblyAI → transcript → UI.

Milestone 2 — LangGraph Full Flow

Objective: convert every meaningful transcript chunk into validated incremental changes to one cumulative structured report.

Step	Node / decision	Behavior
1	receive_chunk	Accept the new transcript event into graph state.
2	understand_chunk	LLM reads new chunk plus current report/context and understands its meaning.
3	determine_updates	Produce section-level updates such as update/append; multiple sections allowed.
4	validate_update	Pydantic validates the structured update; invalid output routes to repair/retry.
5	merge_state	Apply only validated changes to the existing report state.
6	UI event	Send validated updates to the frontend via WebSocket.
7	Repeat	Wait for and process the next transcript chunk.
8	Finish path	Final validation → comment decision → optional one-time AI comment → review.

Comment rule: if the doctor explicitly dictates a comment, store it as clinician-authored. If no clinician comment exists, generate an AI comment only after Finish, using the final validated report. Mark it AI-generated and require clinician review. Never generate comments after every chunk.

Milestone 3 — UI Generation & Enhancement

Objective: turn the working streaming pipeline into a usable clinician-facing MVP.

UI area	MVP behavior
Voice controls	Start/stop, connection status, transcription status.
Live transcript	Show current transcript chunk and compact recent history.
Structured report	Progressively fill Patient, Specimen, Gross Description, Microscopic Description, Diagnosis, Comments.
Field updates	Show validated field changes; do not stream raw LLM tokens into clinical fields.
Comments	Clearly label clinician vs AI-generated; AI comment requires review.
Review/edit	Clinician can edit and approve the final structured report.
Finish/save	Finish performs final validation/comment decision and saves to Supabase.
Errors	Show recoverable STT, WebSocket, validation, and finalization errors.

MVP Folder Structure

Keep the MVP deliberately simple. No Docker, Redis, Celery, Kafka, Kubernetes, microservices, RAG/vector database, authentication/multitenancy, or system-wide OS dictation is required at this stage.

```
pathology-voice-reporting/
■■■ app/
■ ■■■ main.py
■ ■■■ api/routes/reports.py
■ ■■■ api/websocket/voice.py
■ ■■■ core/{config.py,logging.py}
■ ■■■ graph/
■ ■ ■■■ graph.py
■ ■ ■■■ state.py
■ ■ ■■■ nodes/
■ ■ ■ ■■■ receive_chunk.py
■ ■ ■ ■■■ understand_chunk.py
■ ■ ■ ■■■ determine_updates.py
■ ■ ■ ■■■ validate_update.py
■ ■ ■ ■■■ merge_state.py
■ ■ ■ ■■■ generate_comment.py
■ ■ ■ ■■■ finalize_report.py
■ ■ ■■■ routers/{validation_router.py,update_router.py}
■ ■ ■■■ prompts/{extraction.py,comment.py}
■ ■■■ services/
■ ■ ■■■ stt/{base.py,assemblyai.py}
■ ■ ■■■ llm/{base.py,provider.py}
■ ■ ■■■ streaming/{session.py,manager.py}
■ ■■■ schemas/{report.py,voice.py,events.py}
■ ■■■ db/supabase.py
■ ■■■ repositories/report.py
■■■ tests/{test_graph.py,test_report.py,test_websocket.py}
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
```

Final MVP Boundary

In scope: streaming voice, AssemblyAI STT, FastAPI/WebSocket, LangGraph incremental extraction, Pydantic validation, cumulative in-session state, Supabase final persistence, progressive UI, clinician review, and conditional AI comment generation.

Out of scope: autonomous diagnosis/clinical decision-making, distributed infrastructure, RAG, complex auth/multitenancy, audit/versioning, system-wide hotkeys, and production-scale deployment.